

SD1 — Distributed Job Scheduler (Amazon SDE II / Google L4 bar, L5 extensions)

Oracle: top reported HLD prompt. Variants seen: "concurrent job orchestrator with priority + license limits + autoscaling", "cron at scale", "run a job exactly once across N workers". Tags: [hot path] sync user-facing · [async] queued/background · [control plane] config/coordination. ? = likely follow-up at that step, with the answer you give.

Index

0. Time budget
1. Why this is hard
2. Crux
3. Clarifying questions
4. Functional requirements
5. Non-functional requirements
6. Back-of-envelope
7. High-level design
8. API
9. Data model
10. Deep dives (4)
11. Hygiene checklist
12. Operational readiness
13. Pop-up questions
14. Trade-offs + defeaters
15. Wrap-up script
16. Memorization card
17. References

\$0 Time budget

Section	Min
Clarify	4
Why hard + crux	3
FR / NFR	4
BoE	3
HLD + walkthroughs	8
API + schema	4
Deep dives (2 of 4, interviewer picks)	11

Section	Min
Tradeoffs + wrap	4
Slack	4

🗣️ Transition line: "I'll spend 4 min scoping, then design, then I'd like to go deep on execution guarantees and on backpressure — tell me if you'd rather go elsewhere."

§1 Why this is hard

naive: cron table in Postgres → one poller → runs job inline
breaks at: 1 poller dies → nothing runs; 2 pollers → job runs twice

- Naive design → one `jobs` table, one process polls `WHERE next_run <= now()`, executes inline
- First thing that breaks → availability: single poller = single point of failure. The moment you add a second poller for HA you get **duplicate execution** → then throughput: inline execution blocks the poller; at ~1k jobs/min with 1 s jobs you need 17 concurrent executors (assume)
- Minimum primitive that fixes it → **lease** (row-level claim with TTL) — not consensus. Claim the row atomically, own it for N seconds, renew or lose it → why not Raft: you don't need a global order, you need "at most one active owner per job at a time"
- What the user wants → "My job runs when I said, once, and I can see whether it ran"
- Cost of being wrong → duplicate run of a payment job = money; missed run of a cert-rotation job = outage → demands: at-least-once *delivery* + idempotent *execution* + strong consistency on the job's state row

🗣️ "So this is a **lease problem at ~1k/min**, because the failure that matters is two workers thinking they own the same job."

? *Why not just use AWS EventBridge / Airflow?* → "Buy for the trigger, build the execution guarantees. EventBridge gives at-least-once delivery; it does not make my job idempotent or give me priority queues with per-tenant limits. Airflow is a DAG orchestrator with a single scheduler process by default (as of Airflow 2.x, HA scheduler needs a DB with row locks — same lease pattern). I'd use them if scale is <100 jobs/min and no tenant isolation."

§2 Crux

- Hard part: **exactly-once effect from at-least-once execution** — the scheduler can't promise exactly-once; the job + idempotency key does
- Critical success points → atomic claim (no two workers own one run) → lease renewal + fencing token so a paused worker can't write after losing the lease → durable "run record" before execution starts (so a crash is visible, not silent) → bounded queue + priority so a flood from tenant A can't starve tenant B → observability: "did run #123 happen, when, how long, why did it fail"
- Toy vs on-call-runnable → toy: cron + queue. Runnable: dead-letter, retry budget, per-tenant quota, dashboards on lag
- Business metric → system metric → "% of jobs started within SLA window" → `schedule_lag_p99` (due_time → start_time)

💡 "The crux is one-owner-per-run under worker failure, so I'll design around a leased run record with fencing."

§3 Clarifying questions (with assumed answers)

Question	Assumed
Job types? cron, one-shot, delayed?	all three
Scale?	10M jobs defined, 1k runs/min avg, 10× burst top of hour
Duration?	p50 5 s, p99 10 min; max 1 h (assume)
Exactly-once required?	at-least-once execution; jobs must be idempotent; we provide the key
Priority / tenant isolation?	yes, 3 priority classes, per-tenant concurrency cap
Execution: we run code or call a webhook/queue?	we invoke a customer endpoint or enqueue to their queue (control-plane style)
Schedule accuracy?	start within 30 s of due time p99
Multi-region?	single region, multi-AZ; DR later

§4 Functional requirements

- Core → create/update/delete job with schedule (cron or timestamp), payload, priority, retry policy → trigger run at due time; invoke target; record outcome → retry with backoff; DLQ after N → query run history; cancel a job / a run
- Extended (mention, defer) → DAG dependencies; manual re-run; pause/resume; rate-limit per target
- Non-goals → running arbitrary user code in our fleet; sub-second precision; cross-region failover in v1

§5 Non-functional requirements

Requirement	Target	Why	Enforced by
Schedule lag	p99 < 30 s	user promise	sharded scanners, queue depth alarm
API latency	create p99 < 200 ms	control-plane call	Postgres single-row write
Availability (trigger)	99.95% monthly	missed cert rotation = outage	multi-AZ workers, no single scanner
Consistency	strong on run state; eventual on history views	correctness of ownership	Postgres row lock / conditional update
Durability	run record survives 1 AZ loss	audit	Aurora/Postgres multi-AZ sync replica
Throughput	1k/min avg, 10k/min burst	top-of-hour	queue absorbs; workers autoscale
Cost	order of magnitude: \$2–4k/mo at this scale (assume)		one Aurora r6g.large pair + 10–30 small workers + SQS
Security	tenant-scoped IAM; signed webhook	multi-tenant	gateway authz + HMAC on invocation

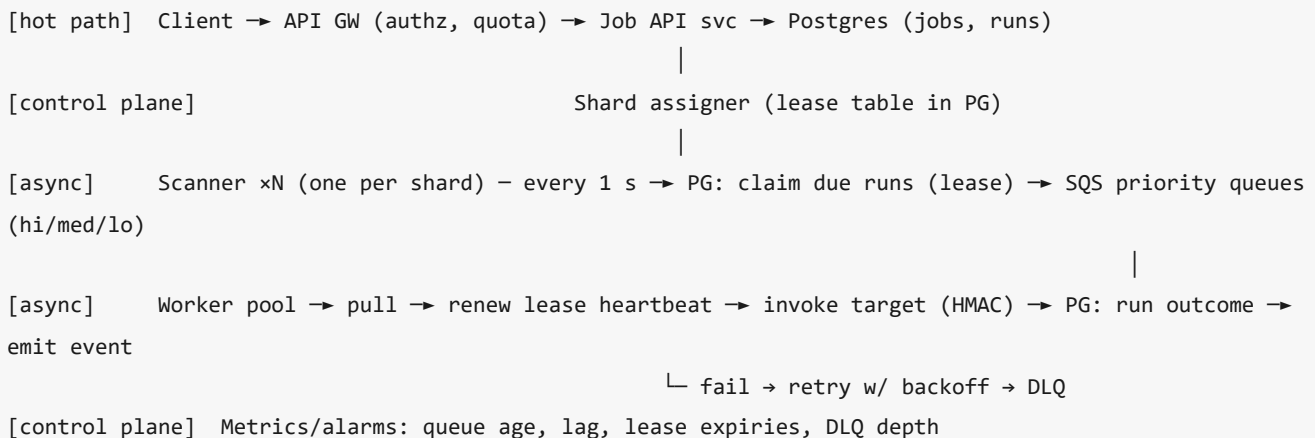
Requirement	Target	Why	Enforced by
Degradation	if invoker down → runs queue, no loss; if DB down → no new triggers, in-flight finish		outbox + DLQ

L5: name the SLO the *customer* signs: "start within 30 s p99" and the internal SLO that protects it: "queue age < 10 s". Also cost of duplicate vs missed run per tenant class — payment tenants get lower lease TTL and stricter idempotency enforcement.

\$6 Back-of-envelope

- Runs: 1k/min = 17/s avg; burst 10k/min = 170/s
- Run record ~1 KB → 1.5M/day → 1.5 GB/day → ~550 GB/yr raw; retain 90 days hot = ~135 GB
- Due-scan: 10M jobs, but only those due in next minute matter; index on (shard, next_run_at) → scan ~1k rows/min, trivial
- Workers: p50 5 s, 170/s burst → 850 concurrent at burst; at p99 10 min tail, need ~1k+ slots (assume); autoscale 50→1000 pods
- Queue: SQS standard handles this without thought; Kafka overkill
- So what: DB write rate is tiny (~20–200/s) → **Postgres is fine**; the fleet size is the cost driver → workers must be cheap and stateless

\$7 High-level design



- Components → API service [hot path]: CRUD on jobs; writes next_run_at → Postgres [hot path]: system of record for jobs + runs; strong consistency boundary → Shard assigner [control plane]: N scanners each lease a shard id (0..K-1) via shard_lease table → Scanner [async]: claims due runs for its shard, enqueues → SQS queues [async]: 3 priority queues; bounded by visibility timeout + DLQ → Workers [async]: stateless; renew lease; invoke; write outcome → Event bus [async]: RunCompleted for downstream/notifications

Primary write — create job

1. Client POST /jobs with Idempotency-Key — waits on 201
2. API GW validates token, checks tenant quota — fails → 403/429

3. Job svc computes `next_run_at`, inserts row (`job`, `next_run_at`, `shard = hash(job_id) % K`) in one txn with idempotency record — fails → 5xx, client retries same key, second insert hits unique constraint → returns original 201
4. Returns 201 with `job_id`

Primary async — trigger a run

1. Scanner for shard `s` (holds `shard_lease` row, TTL 15 s, renews every 5 s) runs every 1 s
2. `UPDATE jobs SET next_run_at = next(cron), version = version+1 WHERE shard = s AND next_run_at <= now() RETURNING ...` inside a txn; inserts `runs(run_id, job_id, due_at, state=SCHEDULED, fence = version)` — this is the atomic claim; another scanner on same shard can't exist (shard lease) and even if it did, the `RETURNING` set is disjoint because the row's `next_run_at` moved — fails mid-txn → rollback, next tick retries; nothing enqueued
3. In the **same txn** write to `outbox(run_id, queue)` — commit
4. Outbox relay publishes to SQS priority queue — on SQS failure, relay retries; outbox row stays until ack → at-least-once to queue
5. Worker receives message, `UPDATE runs SET state=RUNNING, worker=w, lease_until=now()+30s WHERE run_id=? AND state=SCHEDULED` — 0 rows updated → another worker has it → ack message and drop
6. Worker heartbeats `lease_until` every 10 s; invokes target with header `X-Run-Id`, `X-Fence: <fence>`; waits with timeout = job max
7. On success: `UPDATE runs SET state=SUCCEEDED WHERE run_id=? AND fence=?` — if fence mismatch (lease was lost and re-claimed) → discard, log `stale_writer`
8. On failure: retry count < N → `state=SCHEDULED, next_attempt=now()+backoff`, re-enqueue with delay; else → `FAILED`, DLQ
9. Reaper [control plane] every 30 s: `runs WHERE state=RUNNING AND lease_until < now()` → back to `SCHEDULED` with `fence+1`, re-enqueue, metric `lease_expired`

- Consistency boundary: Postgres `runs` row. Everything after the queue is at-least-once.
- Single point of contention: the shard's due-scan query — solved by K shards + index

? *Why scan the DB instead of a delay queue / timing wheel?* → "Because schedules mutate (user edits cron). A DB index on `next_run_at` is the timing wheel and it's trivially editable. SQS delay caps at 15 min (AWS docs) so it can't hold a job due next week. I'd use SQS delay only for the retry backoff."

? *What if a scanner dies?* → "Its shard lease expires in 15 s; another scanner instance grabs it. Worst case 15 s added lag, within the 30 s SLO. That's the number the TTL was chosen from."

L5: the outbox is what makes step 2→4 correct; without it, a commit followed by a failed publish = silent miss. Also: scanners are the only stateful-ish component — make $K > \text{number of instances}$ so rebalancing is just re-leasing, not resharding. Watch for a hot shard if one tenant has 50% of jobs — shard by `job_id`, not `tenant_id`.

\$8 API

```

POST /v1/jobs          Idempotency-Key: uuid
{ "name": "rotate-cert", "schedule": { "cron": "0 */6 * * *" } | { "at": "2026-09-01T00:00:00Z" },
  "target": { "type": "https", "url": "..." } | { "type": "sqs", "arn": "..." },
  "payload": { ... }, "priority": "high|normal|low",
  "retry": { "max": 5, "backoff": "exp", "base_s": 30, "max_s": 3600 },
  "timeout_s": 600, "concurrency_limit": 1 }
→ 201 { "job_id": "...", "next_run_at": "..." }

GET /v1/jobs/{id}      → job + next_run_at
PATCH /v1/jobs/{id}   (If-Match: version) → 200 | 409
DELETE /v1/jobs/{id}   → 204 (cancels future runs; in-flight completes)
POST /v1/jobs/{id}/runs → 202 manual trigger
GET /v1/jobs/{id}/runs?cursor=&limit= → history, cursor pagination
POST /v1/runs/{run_id}/cancel → 202 (best-effort; sets flag worker checks)

```

- Invocation contract (to customer target) → headers: `X-Run-Id`, `X-Job-Id`, `X-Attempt`, `X-Fence`, `X-Signature: HMAC-SHA256(body, tenant_secret)` → customer must dedupe on `X-Run-Id` — documented; this is where exactly-once lives
- Event: `RunCompleted {run_id, job_id, state, started_at, ended_at, attempt, error}` versioned `v1`

? *How does a customer make their handler idempotent?* → "Store `X-Run-Id` in a table with a unique constraint before doing side effects; on conflict return 200. I give them the key; they own the effect."

§9 Data model (Postgres)

```

CREATE TABLE jobs (
  job_id      uuid PRIMARY KEY,
  tenant_id   uuid NOT NULL,
  shard       smallint NOT NULL,          -- hash(job_id) % 64
  schedule    jsonb NOT NULL,
  target      jsonb NOT NULL,
  priority    smallint NOT NULL,
  next_run_at timestamptz,
  version     bigint NOT NULL DEFAULT 0,  -- optimistic lock + fence source
  state       text NOT NULL,              -- ACTIVE | PAUSED | DELETED
  created_at  timestamptz NOT NULL
);

CREATE INDEX jobs_due ON jobs (shard, next_run_at) WHERE state = 'ACTIVE';

CREATE TABLE runs (
  run_id      uuid PRIMARY KEY,
  job_id      uuid NOT NULL REFERENCES jobs,
  due_at      timestamptz NOT NULL,
  state       text NOT NULL,              -- SCHEDULED|RUNNING|SUCCEEDED|FAILED|CANCELLED
  attempt     int NOT NULL DEFAULT 1,
  fence       bigint NOT NULL,
  worker_id   text, lease_until timestamptz,
  started_at  timestamptz, ended_at timestamptz, error text
);

CREATE INDEX runs_lease ON runs (lease_until) WHERE state = 'RUNNING';
CREATE INDEX runs_job   ON runs (job_id, due_at DESC);

CREATE TABLE shard_lease (shard smallint PRIMARY KEY, owner text, lease_until timestamptz);
CREATE TABLE outbox (id bigserial PRIMARY KEY, run_id uuid, queue text, published_at timestamptz);

```

- Why Postgres: write rate ~200/s, need row-level atomic claim + txn with outbox. DynamoDB works too (conditional writes) but loses the multi-row txn simplicity
- Partition key risk: `shard` — 64 shards over ≤ 16 scanners; hot only if one job fires every second (then it's a streaming workload, wrong tool)
- Retention: `runs` 90 days hot → archive to S3 Parquet nightly; payload may contain PII → encrypt column, redact in logs
- Partition `runs` by month for cheap archival

❓ *Why not DynamoDB?* → "Fine choice: conditional `UpdateItem` on `state=SCHEDULED` gives the same claim. I'd flip to it if run writes exceed ~5k/s or I need multi-region active-active. What I lose: the outbox in the same transaction — I'd use DynamoDB Streams as the outbox instead."

\$10 Deep dives

10.1 Execution guarantee: lease + fencing (data/consistency)

- Problem: worker claims run, GC-pauses 45 s, lease expires, reaper reassigns, second worker runs it, first wakes and writes SUCCEEDED → duplicate + corrupted state
- Design: every claim increments `fence`; every write is conditional on fence; the invocation carries the fence so a downstream that cares can reject stale writers

Sequence:

1. Worker A: `UPDATE runs SET state=RUNNING, worker='A', fence=fence+1, lease_until=now()+30s WHERE run_id=? AND state=SCHEDULED RETURNING fence` → `fence=7`
 2. A invokes target with `X-Fence: 7`
 3. A pauses 45 s. Reaper: `UPDATE runs SET state=SCHEDULED WHERE state=RUNNING AND lease_until<now()` → re-enqueue
 4. Worker B claims → `fence=8`, invokes with `X-Fence: 8`
 5. A wakes, `UPDATE runs SET state=SUCCEEDED WHERE run_id=? AND fence=7` → 0 rows → A logs `stale_writer`, acks message, exits
 6. B completes → `... WHERE fence=8` → 1 row → SUCCEEDED
- The target got invoked twice (attempt 7 and 8) — **unavoidable** under at-least-once. Dedupe at target on `X-Run-Id`; if target is fence-aware, reject `fence < last seen`

```
# worker claim + fenced complete (psycopg; autocommit txn per statement)
def claim(cur, run_id, worker_id):
    cur.execute("""
        UPDATE runs SET state='RUNNING', worker_id=%s, fence=fence+1,
            lease_until=now()+interval '30 seconds',
            started_at=coalesce(started_at, now())
        WHERE run_id=%s AND state='SCHEDULED' RETURNING fence""",
        (worker_id, run_id))
    row = cur.fetchone()
    return row[0] if row else None        # None -> someone else owns it

def complete(cur, run_id, fence, state):
    cur.execute("UPDATE runs SET state=%s, ended_at=now() WHERE run_id=%s AND fence=%s",
        (state, run_id, fence))
    return cur.rowcount == 1             # False -> stale writer, drop the result
# Time: O(1) per op – one indexed row update
```

```
// worker claim, Spring JDBC
Optional<Long> claim(UUID runId, String worker) {
    return jdbc.query("""
        UPDATE runs SET state='RUNNING', worker_id=?, fence=fence+1,
            lease_until=now()+interval '30 seconds', started_at=coalesce(started_at, now())
        WHERE run_id=? AND state='SCHEDULED' RETURNING fence""",
        (rs, i) -> rs.getLong(1), worker, runId).stream().findFirst();
}

boolean complete(UUID runId, long fence, String state) {
    return jdbc.update("UPDATE runs SET state=?, ended_at=now() WHERE run_id=? AND fence=?",
        state, runId, fence) == 1;
}
```

- Failure modes: heartbeat thread starves (same GC pause) → lease TTL must exceed max expected pause; choose 30 s TTL / 10 s heartbeat (3 missed = lost)
- Test: kill -STOP a worker mid-run for 60 s; assert exactly one SUCCEEDED, one `stale_writer` metric, target saw two invocations with fences 7 and 8

? *Why not a distributed lock in Redis/ZooKeeper?* → "The DB row is the lock, and it's the same row I have to update anyway. A separate lock service adds a second source of truth and the classic Redlock problem: lock says yes, DB says no. Kleppmann's fencing-token argument applies either way; doing it in one row is simpler."

L5: long jobs (1 h) with 30 s lease means 360 heartbeats — fine. What breaks: 100k concurrent RUNNING rows heartbeating = 10k writes/s to one table → move heartbeat to a separate `leases` table or to Redis with the DB fence as truth. Also decide whether cancel is cooperative (flag checked by worker) or forced (kill container) — cooperative by default; forced needs the fence too.

10.2 Outbox + at-least-once to queue (data/consistency)

- Problem: commit run row, then SQS publish fails → run exists, nothing executes; or publish succeeds, commit fails → phantom execution
- Design: transactional outbox; relay publishes; SQS dedupe not relied on (standard queue)

Sequence:

1. Scanner txn: insert `runs` + insert `outbox` — commit atomically
 2. Relay polls `outbox` WHERE `published_at` IS NULL ORDER BY `id` LIMIT 500 with `FOR UPDATE SKIP LOCKED` (multiple relays safe)
 3. Publishes batch to SQS — on partial failure, only successful ids marked `published_at`
 4. Crash between 3 and 4 → republished on next poll → duplicate message → worker claim step (10.1 step 1) makes the duplicate a no-op
- Why not CDC (Debezium)? Fine at scale; adds Kafka + connector ops. At 200 rows/s the poll relay is boring and sufficient

? *Why SQS standard, not FIFO?* → "FIFO gives 5-minute dedupe and ordering per group; I don't need ordering (runs are independent) and dedupe is already handled by the claim. FIFO throughput cap is 3,000 msg/s with batching (AWS docs) — enough here, but it buys nothing."

L5: outbox table grows; delete published rows older than 1 h; alarm on `oldest_unpublished_age` — that's the metric that catches a stuck relay before users see lag.

10.3 Priority, fairness, backpressure (flow control)

- Problem: tenant A schedules 100k jobs at 00:00; tenant B's single critical job waits behind them
- Design → 3 SQS queues by priority; workers poll high:medium:low at 6:3:1 weighted → per-tenant concurrency cap enforced at claim: `SELECT count(*) FROM runs WHERE tenant=? AND state='RUNNING'` — expensive; instead a Redis counter `INCR tenant:{id}:running` with cap check, decremented on completion, TTL as safety → queue-side: if tenant over cap, worker re-enqueues with 5 s delay (not to DLQ) → API-side: per-tenant job-count and runs/min quota at create time
- Backpressure signal: SQS `ApproximateAgeOfOldestMessage > 10 s` → autoscale workers; `> 60 s` → page

Sequence:

1. Worker polls high queue; empty → medium (weighted loop)
2. Receives run for tenant T; `INCR tenant:T:running` → if `> cap` → `DECR`, change visibility to +5 s, continue
3. Else claim (10.1) → execute → on finish `DECR`
4. Redis down → fail open (no cap) with alarm, because blocking all execution is worse than one tenant bursting

? *Isn't a Redis counter racy?* → "INCR is atomic; the race is a crashed worker never DECR-ing. Mitigate with a periodic reconciliation from the `runs` table (truth) and a TTL on the key."

L5: true fairness needs weighted fair queuing per tenant, which SQS can't do — at that point, one queue per large tenant plus a shared pool, or move to a broker with priority (RabbitMQ) or a DB-backed queue with `ORDER BY priority, due_at` and `SKIP LOCKED` (which is what I'd actually start with under 500/s — it removes SQS entirely).

10.4 Retries, timeouts, DLQ (flow control)

- Policy per job: max attempts, exp backoff base 30 s cap 1 h, jitter $\pm 20\%$
- Classification: 5xx/timeout → retryable; 4xx (except 408/429) → terminal → FAILED immediately, no retry burn
- Timeout: worker enforces `timeout_s`; on expiry cancels HTTP call, marks attempt failed, retry
- DLQ: after max attempts → `FAILED`, message to DLQ, `RunCompleted{state:FAILED}` event; console shows error
- Poison pill: a job whose target always 500s consumes retry budget then stops — bounded by max attempts $\times$ backoff; a job whose `scanner` insert always fails (bad schedule) → mark job `PAUSED` with reason after 3 scanner errors

```
import random

def backoff(attempt, base_s=30, cap_s=3600):
    ms = min(cap_s, base_s * (1 << min(attempt, 20)))
    return ms + random.uniform(-ms / 5, ms / 5)    # exp + full-range  $\pm 20\%$  jitter
# Time: O(1)
```

```

Duration backoff(int attempt, Duration base, Duration cap) {
    long ms = Math.min(cap.toMillis(), base.toMillis() * (1L << Math.min(attempt, 20)));
    long jitter = ThreadLocalRandom.current().nextLong(-ms/5, ms/5 + 1);
    return Duration.ofMillis(ms + jitter);
}

```

L5: retry budget per tenant (e.g., retries $\leq$ 10% of successful runs over 1 min) prevents a broken tenant from turning the fleet into a retry storm — the AWS Builders' Library "timeouts, retries, and backoff with jitter" argument.

§11 Hygiene checklist

Concern	Default	Where it bites here
AuthN/Z	OIDC at gateway; tenant-scoped policies	job create/read across tenants; HMAC on invocation
Idempotency	key on POST; claim-by-state on runs	duplicate create; duplicate message
Timeouts/retries	per-hop; exp+jitter; retry budget	target invocation; SQS publish
Rate limiting	token bucket at GW; per-tenant runs/min	top-of-hour floods
Circuit breaker	per target host; skip + reschedule when open	one dead customer endpoint eating workers
Queues/backpressure	3 priority SQS; age alarm; DLQ	tenant starvation
Caching	none on hot path; job def cached in worker 60 s	stale target URL after PATCH → read-through on fence mismatch
Consistency	strong on jobs/runs; eventual on history/metrics	duplicate invoke is the accepted eventual case
Storage/partitioning	shard on job_id; runs partitioned by month	hot tenant
Replication/failover	Aurora multi-AZ, RPO 0, RTO ~1 min (AWS docs order of magnitude)	scanners reconnect; leases survive
Observability	lag, queue age, lease_expired, stale_writer, DLQ depth, per-tenant success rate	
Deploy/rollback	canary workers; schema migrations expand→migrate→contract	fence column add is additive
Retention/PII	90 d hot; payload encrypted; redact logs	
Multi-tenancy	row-level tenant_id; quotas; per-tenant cap	noisy neighbor

§12 Operational readiness

- Capacity: bottleneck = worker concurrency at p99 duration. Headroom 2× burst. Trigger: queue age > 10 s or running/slots > 70%
- Runbook → **Lag rising, queue age high** → dashboard: workers busy? → scale workers; if workers idle → relay stuck → check outbox oldest age → restart relay → **lease_expired spiking** → workers pausing (GC/OOM)? → check heap; raise TTL temporarily; page if stale_writer > 0 → **DLQ growing for one tenant** → their target down → circuit open; notify tenant; don't touch fleet

- Backfill/migration: add column additive; re-shard by doubling K with `shard = hash % 128` computed lazily on next scan — old shards drain
- Failure/fallback: DB down → API 503, workers finish in-flight, no new triggers (safe); SQS down → outbox accumulates, alarm; Redis down → caps fail open
- Testing: load test 10k/min for 10 min with 20% failing targets; chaos: SIGSTOP workers, kill scanner, partition DB for 30 s; nightly reconciler: `runs SCHEDULED older than 5 min` = bug

§13 Pop-up questions

- *Why isn't this just Postgres + `SKIP LOCKED`, no SQS?* → Under ~500 runs/s, it can be — workers `SELECT ... FOR UPDATE SKIP LOCKED ORDER BY priority, due_at LIMIT 1`. I add SQS when I want workers decoupled from DB connections (1000 workers × pool = connection storm) and delayed retries for free.
- *Two scanners on one shard?* → Shard lease in DB; plus the claim query is disjoint by construction. Belt and braces.
- *Clock skew?* → Scanners use DB `now()`, not local clock. Workers' `lease_until` also DB time.
- *Job due every second?* → That's a stream, not a schedule; cap min interval at 60 s; offer a "long-running" job type instead.
- *Cron in which timezone/DST?* → Store tz per job; compute next in tz; DST gap → skip, overlap → run once. Say you'd use a library (cron-utils) not hand-roll.
- *Exactly-once?* → Not offered. At-least-once + run_id for dedupe. Anyone who says exactly-once end-to-end is hiding the dedupe somewhere.
- *Leader dies mid-scan?* → Txn rolls back; lease expires ≤15 s; next scanner sees the same due rows.
- *10× scale (100k/min)?* → DB writes 2k/s still fine on one Aurora writer; shards 64→512; move heartbeats off the runs table; consider DynamoDB for runs.
- *Multi-region?* → Active-passive: Aurora global DB, scanners in one region only (leases prevent double-run); failover = promote + start scanners. Active-active needs jobs pinned to home region.
- *How do you cancel a running job?* → Cooperative flag in `runs`; worker checks between heartbeats; hard kill via container stop after grace, with fence so its late writes are rejected.

§14 Trade-offs

Decision	Chosen	Alternative	Flips when
Truth store	Postgres	DynamoDB	> 5k writes/s or active-active regions
Trigger	DB scan on index	delay queue / timing wheel	schedules immutable and < 15 min horizon
Ownership	DB row lease + fence	ZooKeeper/Redis lock	you already run ZK and need locks elsewhere
Queue	SQS standard	DB SKIP LOCKED	< 500/s and want zero infra
Priority	3 queues weighted	single queue	no starvation risk
Exec model	invoke customer target	run code in our fleet	product is a code runner (then sandboxing dominates)

- Defeaters → "Use Kafka for the job queue" — Kafka has no per-message delay, no per-message ack; retries block the partition. Wrong tool for a work queue → "Exactly-once via SQS FIFO dedupe" — dedupe window is 5 min; a retry after 6 min duplicates. Idempotency must live at the effect → "One

global leader schedules everything" — leader is a throughput ceiling and a failover gap; shard leases give you N leaders with the same safety

§15 Wrap-up (2 min)

"Crux: one owner per run under worker failure. Design: Postgres runs table as the lease, fence on every write, outbox to SQS priority queues, stateless workers that heartbeat and invoke customer targets with a run id for dedupe. NFRs: 30 s lag p99 via sharded scanners with 15 s leases; 99.95% via multi-AZ and no single scanner; at-least-once with idempotency key, no exactly-once claim. MVP0: single scanner, one queue, no priority, fence + outbox from day one — those are the parts you can't retrofit. Week-one metrics: schedule lag, queue age, lease_expired, stale_writer, DLQ. At 10×: more shards, heartbeats off the hot table, DynamoDB for runs."

§16 Memorization card

- Pitch: leased run records in Postgres, fenced writes, outbox → priority queues, idempotent invocation
- Why hard: two workers must never both own a run, and the scheduler can't make the job idempotent
- 5 numbers: 17/s avg, 170/s burst · 30 s lag SLO · lease 30 s / heartbeat 10 s · shard lease 15 s · 64 shards
- 3 failures: GC-pause stale writer (fence) · commit-then-publish loss (outbox) · tenant flood (priority + caps)
- 3 defeaters: Kafka as work queue · FIFO dedupe = exactly-once · single global leader
- MVP0 one scanner + one queue + fence + outbox → MVP1 priority + caps + DLQ → MVP2 multi-region, DAGs
- Done when: can draw HLD with tags in 4 min; write the claim SQL from memory; explain fence in 30 s

§17 References

- github.com/apache/airflow — scheduler `SKIP LOCKED` job claiming (real code)
- github.com/hatchet-dev/hatchet — Postgres-backed task queue, outbox pattern
- Kleppmann, "How to do distributed locking" (fencing tokens)
- AWS Builders' Library, "Timeouts, retries, and backoff with jitter"
- Microservices.io, "Transactional outbox"